

CHƯƠNG 11: NHẬN DẠNG LOÀI HOA

Giới thiệu

Cách tiếp cận truyền thống của con người để phân loại thực vật là so sánh màu sắc và hình dạng của lá, nhưng bằng cách sử dụng phân tích Trí tuệ nhân tạo, ta có thể thu được kết quả nhanh hơn và chi tiết hơn thông qua việc phân tích hình thái của lá và cung cấp thêm chi tiết về các đặc tính của lá. Nhận dạng loài là một phần rất quan trọng của nông nghiệp và có hàng ngàn loài thực vật mà một người không thể nhận dạng hết được, và ngay cả khi làm vậy, cũng sẽ mất rất nhiều thời gian.

Trong chương này, chúng ta sẽ thảo luận và xây dựng một dự án liên quan đến việc nhận dạng các loài hoa bằng hình ảnh. Trong dự án này, chúng ta sẽ sử dụng các nhánh khác nhau của Trí tuệ nhân tạo như học máy (machine learning), học sâu (deep learning) và thị giác máy tính (computer vision) để xây dựng một dự án tuyệt vời như vậy. Nào, chúng ta hãy bắt đầu

Cài đặt các Gói (Packages)

Trong dự án Nhận dạng loài này, chúng ta đã sử dụng rất nhiều thư viện cần được cài đặt trước khi triển khai. Lệnh để cài đặt các thư viện này là:

- Warnings- pip install pytest-warnings/warnings
- NumPy- pip install Numpy
- Pandas- pip install pandas
- Matplotlib- pip install matplotlib
- Seaborn- pip install seaborn
- Tqdm- pip install tqdm
- Pillow/PIL- pip install pillow
- TensorFlow- pip install TensorFlow
- Keras- pip install keras

Đối với IDE Python: Nhập các lệnh này trên command prompt trong Windows. Đối với Anaconda: Nhập các lệnh này trên Anaconda Prompt

Nếu bạn đang sử dụng bất kỳ Môi trường hoặc IDE nào khác, bạn phải chỉ định đường dẫn cho python và bạn có thể thực hiện cài đặt.

Dự án này được triển khai trong Google Research Collaboratory. Và bạn có thể tải xuống bản sao của mã nguồn từ liên kết này

Bộ dữ liệu (Dataset)

Trong dự án này, chúng tôi sử dụng bộ dữ liệu từ Kaggle có 5 loại hoa là cúc họa mi (daisy), bồ công anh (dandelion), hoa hồng (rose), hoa hướng dương (sunflower) và hoa

tulip, tổng cộng bao gồm 4242 hình ảnh với kích thước 320x240 pixel. Các hình ảnh trong bộ dữ liệu được lấy từ dữ liệu của Flickr, Google Images và Yandex Images. Những hình ảnh này có thể được sử dụng để nhận dạng các loài thực vật liên quan.

Bạn có thể tải xuống bộ dữ liệu tương tự từ liên kết này

Triển khai:

Lưu ý: Dự án này hoàn toàn được triển khai trên Google Research Collaboratory mà bạn có thể truy cập bằng liên kết này và bạn có thể tự do sử dụng bất kỳ IDE nào khác để triển khai dự án.

Bước 1: Nhập (Import) tất cả các Thư viện

Lưu ý: Ký hiệu # đại diện cho chú thích (comment) trong dự án, giúp bạn hiểu rõ hơn về mã nguồn.

```
# Bỏ qua các cảnh báo
```

```
import warnings
```

```
warnings.filterwarnings('always')
```

```
warnings.filterwarnings('ignore')
```

```
# trực quan hóa và thao tác dữ liệu
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib import style
```

```
import seaborn as sns
```

```
# cấu hình
```

```
# đặt matplotlib ở chế độ nội tuyến (inline) và hiển thị biểu đồ bên dưới ô tương ứng.
```

```
% matplotlib inline
```

```
style.use('fivethirtyeight')
```

```
sns.set(style='whitegrid',color_codes=True)
```

```
# lựa chọn mô hình
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.model_selection import KFold
```

```
from sklearn.metrics import accuracy_score, precision
```

```
score, recall_score, confusion_matrix, roc_curve,
```

```
roc_auc_score
```

```
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.preprocessing import LabelEncoder
```

```

# tiền xử lý.
from keras.preprocessing.image import ImageDataGenerator

# Đây là các thư viện học sâu có trong Keras. Nếu bạn đã cài đặt Keras, tất cả chúng sẽ
# được cài đặt tự động.
from keras import backend as K
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam, SGD, Adagrad, Adadelta, RMSprop
from keras.utils import to_categorical

# chúng ta sử dụng các thư viện này đặc biệt để xây dựng Mạng nơ-ron tích chập (CNN)
# sẽ được dùng để nhận dạng các loài hoa.
from keras.layers import Dropout, Flatten, Activation
from keras.layers import Conv2D, MaxPooling2D,
BatchNormalization

import tensorflow as tf
import random as rn

# ở đây chúng ta sử dụng thư viện OpenCV để cung cấp đầu vào cho mô hình đã huấn
# luyện
# và đặc biệt là để thao tác các hình ảnh đã nén (zipped) và lấy các mảng NumPy chứa
# giá trị pixel của hình ảnh.
import cv2
import numpy as np
from tqdm import tqdm
import os
from random import shuffle
from zipfile import ZipFile
from PIL import Image

```

Bước 2: Gắn (Mount) drive để lấy bộ dữ liệu

Nếu bạn đang sử dụng Google Research Collaboratory, bạn có thể tải xuống bộ dữ liệu từ liên kết này và tải nó lên thư mục “Colab Notebooks” trong Google Drive của bạn, sau đó bạn có thể liên kết nó bằng mã này

```

from google.colab import drive
drive.mount('/gdrive')

```

Kết quả: Go to this URL in a browser:

https://accounts.google.com/o/oauth2/auth?client_id=94.y Enter your authorization code: (Nhập mã ủy quyền của bạn:)

Mounted at /gdrive (Đã gắn tại /gdrive)

Đây là mã để gắn drive của bạn với Google Research Collaboratory, nó sẽ yêu cầu bạn xác thực. Bạn có thể làm điều này bằng tài khoản Gmail của mình chỉ bằng cách làm theo các bước đơn giản. Như bạn có thể thấy trong kết quả, Collaboratory sẽ yêu cầu bạn xác minh bằng cách nhấp vào liên kết đã cho, sau đó nó sẽ được gắn sau khi bạn cấp quyền xác thực.

```
with open ('/gdrive/My Drive/foo.txt', 'w') as f:  
    f.write ('Hello Google Drive!')  
!cat '/gdrive/My Drive/foo.txt'
```

Kết quả: Hello Google Drive!

Đây là mã giúp bạn kiểm tra xem drive của mình đã được gắn với Collaboratory hay chưa.

Sau khi làm điều này, chúng ta sẽ truy cập trực tiếp vào bộ dữ liệu đã được tải lên drive và giải nén nó để truy cập các tệp mà chúng ta cần trong dự án này.

Bước 3: Chuẩn bị dữ liệu sẵn sàng cho Mô hình

Ở đây, chúng ta đang tạo một mảng hình ảnh X và Z sẽ lưu trữ tất cả các hình ảnh dưới dạng hình ảnh huấn luyện và xác thực để mô hình CNN huấn luyện và được kiểm tra với dữ liệu thực tế.

```
X=[]  
Z=[]  
IMG_SIZE = 150 #gán kích thước hình ảnh không đổi cho dự án  
FLOWER_Daisy_DIR='drive/Colab Notebooks/flower recog/flowers/  
daisy'  
FLOWER_SUNFLOWER_DIR='drive/Colab Notebooks/flower recog/  
flowers/sunflower'  
FLOWER_TULIP_DIR='drive/Colab Notebooks/flower recog/flowers/  
tulip'  
FLOWER_DANDI_DIR='drive/Colab Notebooks/flower recog/flowers/  
dandelion'  
FLOWER_ROSE_DIR='drive/Colab Notebooks/flower recog/flowers/  
rose'
```

Tạo một hàm để gán nhãn cho các hình ảnh trong dữ liệu, sẽ được sử dụng tiếp cho mục đích huấn luyện.

```
def assign_label (img, flower_type):  
    return flower_type
```

ở đây sau khi gán nhãn, chúng ta sẽ tạo dữ liệu huấn luyện riêng biệt từ mỗi loại hoa

```
def make_train_data (flower_type, DIR):
    for img in tqdm (os.listdir (DIR)):
        label = assign_label (img, flower_type)

        path = os.path.join (DIR,img)
        img = cv2.imread (path,cv2.IMREAD_COLOR)
        img = cv2.resize (img, (IMG_SIZE, IMG_SIZE))

        X.append (np.array (img))
        Z.append (str (label))
```

```
# Dữ liệu huấn luyện cho hoa cúc họa mi (Daisy)
make_train_data ('Daisy', FLOWER_DAIKY_DIR)
print (len (X))
```

Kết quả: 100% 769/769 [00:07<00:00, 101.93it/s] 769

```
#Dữ liệu huấn luyện cho hoa hướng dương (Sunflower)
make_train_data ('Sunflower', FLOWER_SUNFLOWER_DIR)
print (len (X))
```

Kết quả: 100% 766/766 [00:07<00:00, 176.83it/s] 1535

```
#Dữ liệu huấn luyện cho hoa Tulip
make_train_data ('Tulip', FLOWER_TULIP_DIR)
print (len (X))
```

Kết quả: 100% 991/991 [00:08<00:00, 116.01it/s] 2526

```
#Dữ liệu huấn luyện cho bồ công anh (Dandelion)
make_train_data ('Dandelion', FLOWER_DANDI_DIR)
print (len (X))
```

Kết quả: 100% 1052/1052 [00:09<00:00, 114.67it/s] 3578

```
#Dữ liệu huấn luyện cho hoa hồng (Rose)
make_train_data ('Rose', FLOWER_ROSE_DIR)
print (len (X))
```

Kết quả: 100% 784/784 [00:06<00:00, 116.67it/s] 4362

Đây là cách dữ liệu huấn luyện cho tất cả các loài hoa đã sẵn sàng để mô hình được huấn luyện.

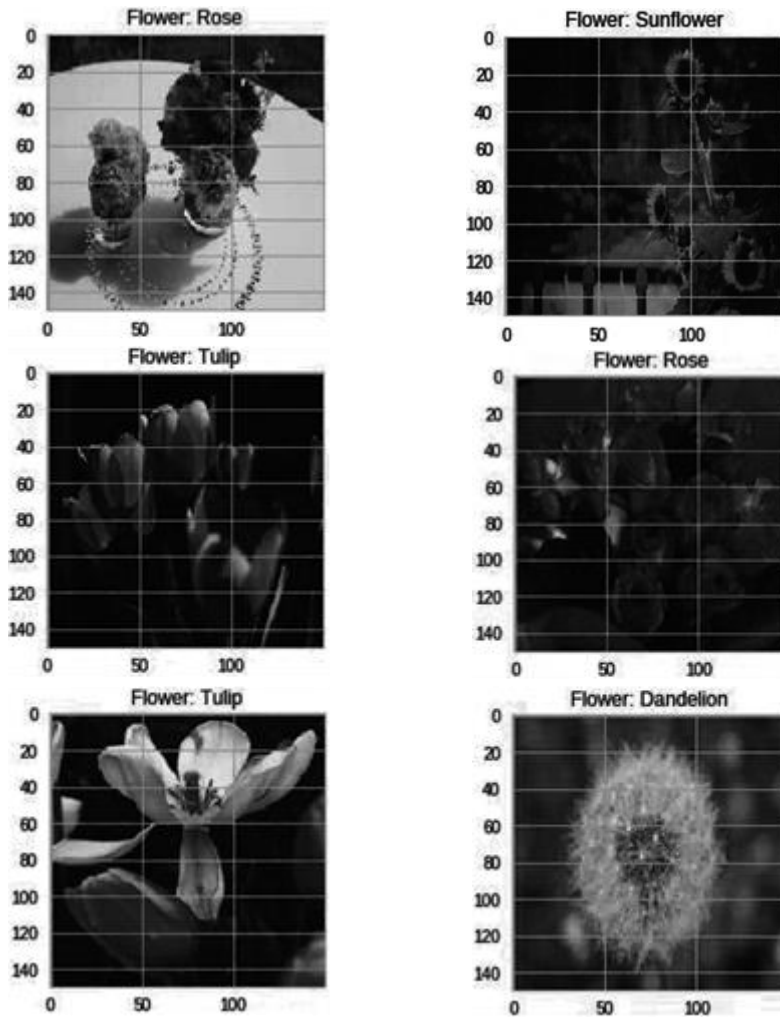
Bước 4: Trực quan hóa dữ liệu

Sau khi chuẩn bị dữ liệu để huấn luyện, chúng ta sẽ trực quan hóa dữ liệu đã chuẩn bị bằng Matplotlib và OpenCV.

```
fig,ax=plt.subplots (5,2)
fig.set_size_inches (15,15)
for i in range (5):
    for j in range (2):
        l=mn.randint (0,len (Z))
        ax[i,j] .imshow (X[l])
        ax[i,j] .set_title ('Flower: '+Z[l])

plt.tight_layout ()
```

Kết quả: như trong Hình 11.1



Bước 5: Gán nhãn dữ liệu và chia thành dữ liệu huấn luyện (training) và dữ liệu xác thực (validation)

#gán nhãn

```
le = LabelEncoder ()
```

```
Y = le.fit_transform (Z)
```

```
Y = to_categorical (Y,5)
```

```
X = np.array (X)
```

```
X = X/255
```

#Sau khi gán nhãn, chúng ta sẽ chia dữ liệu thành các Tập Huấn luyện và Xác thực
`x_train,x_test,y_train,y_test=train_test_split (X, Y, test_size=0.25, random_state=42)`

#Đặt các mầm ngẫu nhiên (random seeds)

```
np.random.seed (42)
```

```
rn.seed (42)
```

```
tf.set_random_seed (42)
```

Bước 6: Xây dựng Mô hình

Để xây dựng mô hình, tôi đã sử dụng Keras Sequential, với nó, việc xây dựng bất kỳ mô hình nào có đầu vào là nhiều hình ảnh để trích xuất đặc trưng và phân loại chúng trở nên rất dễ dàng.

Trong quá trình tạo mô hình, tôi đã tạo bốn lớp tích chập (convolutional layer). Ở lớp đầu tiên, chúng ta có 32 bộ lọc (filters), ma trận kernel 5x5 với đệm (padding) và hàm kích hoạt (activation function) ReLU cùng với kích thước đầu vào của hình ảnh.

Ở lớp thứ hai, chúng ta có 64 bộ lọc, ma trận kernel 3x3 với đệm và hàm kích hoạt ReLU cùng với kích thước đầu vào của hình ảnh.

Ở lớp thứ ba và thứ tư, chúng ta có 96 bộ lọc, ma trận kernel 3x3 với đệm và hàm kích hoạt ReLU cùng với kích thước đầu vào của hình ảnh.

Theo sau tất cả các lớp tích chập này, chúng ta sẽ có một lớp gộp cực đại (max pooling layer) giúp chúng ta lấy được các đặc trưng tối đa từ các bộ lọc.

Giữa các lớp, chúng ta có lớp “flatten ()” (làm phẳng) luôn đóng vai trò là kết nối giữa lớp tích chập và lớp kết nối đầy đủ (dense layer).

Và cuối cùng, chúng ta có các lớp đầu ra kết nối đầy đủ, tức là các lớp dense với một hàm kích hoạt.

bắt đầu lập mô hình bằng CNN.

```
model = Sequential ()
```

```
model.add (Conv2D (filters = 32, kernel_size = (5,5), padding = 'Same', activation = 'relu', input_shape = (150,150,3)))
```

```

model.add (MaxPooling2D (pool_size= (2,2)))

model.add (Conv2D (filters = 64, kernel_size = (3,3),padding = 'Same', activation =
'relu'))
model.add (MaxPooling2D (pool_size = (2,2), strides = (2,2)))

model.add (Conv2D (filters = 96, kernel_size = (3,3), padding = 'Same',activation
='relu'))
model.add (MaxPooling2D (pool_size = (2,2), strides= (2,2)))

model.add (Conv2D (filters = 96, kernel_size = (3,3),padding = 'Same', activation
='relu'))
model.add (MaxPooling2D (pool_size = (2,2), strides = (2,20)))

model.add (Flatten ())
model.add (Dense (512))
model.add (Activation ('relu'))
model.add (Dense (5, activation = "softmax"))

#sau khi xây dựng mô hình, chúng ta sẽ khai báo số lượng epochs và kích thước lô (batch
size)
# và chúng ta đang sử dụng một LR annealer (bộ giảm tốc độ học) để cố định tốc độ học
(learning rate) của chúng ta
# sau một khoảng thời gian nhất định trong khi biên dịch mô hình.

#sử dụng một LR Annealer
batch_size = 128
epochs = 50

from keras.callbacks import ReduceLROnPlateau
red_lr = ReduceLROnPlateau (monitor='val_acc',
patience = 3,verbose = 1,factor = 0.1)

#chúng ta đang thực hiện Tăng cường Dữ liệu (Data Augmentation) ở đây để ngăn chặn
việc Overfitting (học vẹt)
datagen = ImageDataGenerator (
    featurewise_center=False, # đặt trung bình đầu vào về 0 trên toàn bộ bộ dữ liệu
    samplewise_center=False, # đặt trung bình của mỗi mẫu về 0
    featurewise_std_normalization=False, # chia đầu vào cho độ lệch chuẩn (std) của bộ
dữ liệu
    samplewise_std_normalization=False, # chia mỗi đầu vào cho độ lệch chuẩn của nó
    zca_whitening=False, # áp dụng làm trắng ZCA (ZCA whitening)
    rotation_range = 10, # xoay ảnh ngẫu nhiên trong phạm vi (độ, 0 đến 180)
    zoom_range = 0.1, # Thu phóng hình ảnh ngẫu nhiên

```



```

width_shift_range = 0.2, # dịch chuyển ảnh ngẫu nhiên theo chiều ngang (phần trăm
của tổng chiều rộng)
height_shift_range = 0.2, # dịch chuyển ảnh ngẫu nhiên theo chiều dọc (phần trăm của
tổng chiều cao)
horizontal_flip=True, # lật ảnh ngẫu nhiên theo chiều ngang
vertical_flip=False) # lật ảnh ngẫu nhiên theo chiều dọc

```

```

datagen.fit(x_train)

```

bây giờ chúng ta sẽ Biên dịch (Compiling) Mô hình & Xem Tóm tắt (Summary) của Mô hình

```

model.compile(optimizer=Adam(lr=0.001), loss='categorical_crossentropy',
metrics=['accuracy'])
model.summary()

```

Kết quả:

Layer (type)	Output Shape	Param #
=====		
==		
conv2d_1 (Conv2D)	(None, 150, 150, 32)	2432
max_pooling2d_1 (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_2 (Conv2D)	(None, 75, 75, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_3 (Conv2D)	(None, 37, 37, 96)	55392
max_pooling2d_3 (MaxPooling2D)	(None, 18, 18, 96)	0
conv2d_4 (Conv2D)	(None, 18, 18, 96)	83040
max_pooling2d_4 (MaxPooling2D)	(None, 9, 9, 96)	0
flatten_1 (Flatten)	(None, 7776)	0
dense_1 (Dense)	(None, 512)	3981824
activation_1 (Activation)	(None, 512)	0
dense_2 (Dense)	(None, 5)	2565
=====		

==

Total params: 4,143,749

Trainable params: 4,143,749

Non-trainable params: 0

Bước 7: Huấn luyện Mô hình với bộ dữ liệu

Sau khi biên dịch mô hình, chúng ta sẽ huấn luyện mô hình của mình với dữ liệu huấn luyện (training data) mà chúng ta đã gán nhãn và tách ra trước đó.

```
History = model.fit_generator (datagen.flow (x_train,y_train,  
batch_size=batch_size),  
epochs = epochs, validation_data = (x_test,y_test),  
verbose = 1, steps_per_epoch = x_train.shape[0] // batch_size)
```

```
# model.fit (x_train,y_train, epochs=epochs,batch_size=batch_size, validation_data =  
(x_test,y_test))
```

Kết quả: Epoch 1/50 25/25 [=====] - 23s 903ms/step
- loss: 1.4743 - acc: 0.3588 - val_loss: 1.2462 - val_acc: 0.5069 Epoch 2/50 25/25
[=====] - 20s 808ms/step - loss: 1.1573 - acc: 0.5212
- val_loss: 1.1553 - val_acc: 0.5252 Epoch 3/50 25/25
[=====] - 20s 812ms/step - loss: 1.0605 - acc: 0.5745
- val_loss: 1.0670 - val_acc: 0.6013 Epoch 4/50 25/25
[=====] - 21s 820ms/step - loss: 1.0024 - acc: 0.5956
- val_loss: 0.9759 - val_acc: 0.6114 Epoch 5/50 25/25
[=====] - 20s 795ms/step - loss: 0.9267 - acc: 0.6437
- val_loss: 0.9823 - val_acc: 0.6205 Epoch 6/50 25/25
[=====] - 20s 784ms/step - loss: 0.9096 - acc: 0.6477
- val_loss: 0.9942 - val_acc: 0.6004 Epoch 7/50 25/25
[=====] - 20s 807ms/step - loss: 0.8594 - acc: 0.6677
- val_loss: 0.9092 - val_acc: 0.6581 Epoch 8/50 25/25
[=====] - 20s 812ms/step - loss: 0.8468 - acc: 0.6779
- val_loss: 0.8853 - val_acc: 0.6783 Epoch 9/50 25/25
[=====] - 20s 810ms/step - loss: 0.7789 - acc: 0.7019
- val_loss: 0.8087 - val_acc: 0.6856 Epoch 10/50 25/25
[=====] - 20s 798ms/step - loss: 0.7613 - acc: 0.7089
- val_loss: 0.8019 - val_acc: 0.6939 Epoch 11/50 25/25
[=====] - 20s 795ms/step - loss: 0.7660 - acc: 0.6997
- val_loss: 0.8029 - val_acc: 0.7030 Epoch 12/50 25/25
[=====] - 20s 794ms/step - loss: 0.7742 - acc: 0.7022
- val_loss: 0.9206 - val_acc: 0.6480 Epoch 13/50 25/25
[=====] - 20s 794ms/step - loss: 0.7367 - acc: 0.7154

- val_loss: 0.7945 - val_acc: 0.6929 Epoch 14/50 25/25
[=====] - 19s 778ms/step - loss: 0.7020 - acc: 0.7354
- val_loss: 0.7705 - val_acc: 0.6948 Epoch 15/50 25/25
[=====] - 20s 807ms/step - loss: 0.6742 - acc: 0.7325
- val_loss: 0.7577 - val_acc: 0.7140

Epoch 16/50 25/25 [=====] - 20s 781ms/step - loss:
0.6758 - acc: 0.7418 - val_loss: 0.7766 - val_acc: 0.7131 Epoch 17/50 25/25
[=====] - 20s 810ms/step - loss: 0.6605 - acc: 0.7466
- val_loss: 0.7200 - val_acc: 0.7269 Epoch 18/50 25/25
[=====] - 20s 793ms/step - loss: 0.6453 - acc: 0.7500
- val_loss: 0.7373 - val_acc: 0.7186 Epoch 19/50 25/25
[=====] - 20s 808ms/step - loss: 0.6381 - acc: 0.7585
- val_loss: 0.7292 - val_acc: 0.7269 Epoch 20/50 25/25
[=====] - 20s 795ms/step - loss: 0.6110 - acc: 0.7673
- val_loss: 0.7906 - val_acc: 0.7131 Epoch 21/50 25/25
[=====] - 20s 795ms/step - loss: 0.6187 - acc: 0.7629
- val_loss: 0.7009 - val_acc: 0.7296 Epoch 22/50 25/25
[=====] - 20s 797ms/step - loss: 0.5574 - acc: 0.7865
- val_loss: 0.6847 - val_acc: 0.7599 Epoch 23/50 25/25
[=====] - 20s 799ms/step - loss: 0.5884 - acc: 0.7788
- val_loss: 0.6892 - val_acc: 0.7415 Epoch 24/50 25/25
[=====] - 20s 796ms/step - loss: 0.5553 - acc: 0.7869
- val_loss: 0.7059 - val_acc: 0.7360 Epoch 25/50 25/25
[=====] - 20s 797ms/step - loss: 0.5731 - acc: 0.7867
- val_loss: 0.6628 - val_acc: 0.7479 Epoch 26/50 25/25
[=====] - 20s 799ms/step - loss: 0.5141 - acc: 0.7989
- val_loss: 0.6582 - val_acc: 0.7681 Epoch 27/50 25/25
[=====] - 20s 800ms/step - loss: 0.4982 - acc: 0.8122
- val_loss: 0.8243 - val_acc: 0.7232 Epoch 28/50 25/25
[=====] - 20s 801ms/step - loss: 0.5484 - acc: 0.7898
- val_loss: 0.6767 - val_acc: 0.7562 Epoch 29/50 25/25
[=====] - 20s 800ms/step - loss: 0.4928 - acc: 0.8129
- val_loss: 0.7491 - val_acc: 0.7498 Epoch 30/50 25/25
[=====] - 20s 799ms/step - loss: 0.4768 - acc: 0.8238
- val_loss: 0.6758 - val_acc: 0.7544 Epoch 31/50 25/25
[=====] - 20s 795ms/step - loss: 0.4678 - acc: 0.8219
- val_loss: 0.7470 - val_acc: 0.7415

Epoch 32/50 25/25 [=====] - 20s 798ms/step - loss: 0.4604 - acc: 0.8260 - val_loss: 0.6842 - val_acc: 0.7489 Epoch 33/50 25/25 [=====] - 20s 796ms/step - loss: 0.4755 - acc: 0.8263 - val_loss: 0.7710 - val_acc: 0.7589 Epoch 34/50 25/25 [=====] - 20s 800ms/step - loss: 0.4409 - acc: 0.8345 - val_loss: 0.6872 - val_acc: 0.7782 Epoch 35/50 25/25 [=====] - 20s 807ms/step - loss: 0.4372 - acc: 0.8378 - val_loss: 0.6577 - val_acc: 0.7626 Epoch 36/50 25/25 [=====] - 20s 810ms/step - loss: 0.3909 - acc: 0.8534 - val_loss: 0.6526 - val_acc: 0.7910 Epoch 37/50 25/25 [=====] - 20s 784ms/step - loss: 0.3928 - acc: 0.8558 - val_loss: 0.7110 - val_acc: 0.7663 Epoch 38/50 25/25 [=====] - 20s 793ms/step - loss: 0.3955 - acc: 0.8530 - val_loss: 0.7136 - val_acc: 0.7773 Epoch 39/50 25/25 [=====] - 20s 808ms/step - loss: 0.3658 - acc: 0.8659 - val_loss: 0.6593 - val_acc: 0.7773 Epoch 40/50 25/25 [=====] - 20s 798ms/step - loss: 0.3916 - acc: 0.8588 - val_loss: 0.6378 - val_acc: 0.7782 Epoch 41/50 25/25 [=====] - 20s 797ms/step - loss: 0.3722 - acc: 0.8565 - val_loss: 0.7199 - val_acc: 0.7736 Epoch 42/50 25/25 [=====] - 20s 782ms/step - loss: 0.3746 - acc: 0.8533 - val_loss: 0.6931 - val_acc: 0.7791 Epoch 43/50 25/25 [=====] - 20s 795ms/step - loss: 0.3255 - acc: 0.8787 - val_loss: 0.6562 - val_acc: 0.8029 Epoch 44/50 25/25 [=====] - 20s 809ms/step - loss: 0.3306 - acc: 0.8738 - val_loss: 0.7102 - val_acc: 0.7635 Epoch 45/50 25/25 [=====] - 20s 780ms/step - loss: 0.2981 - acc: 0.8809 - val_loss: 0.7033 - val_acc: 0.7929 Epoch 46/50 25/25 [=====] - 20s 809ms/step - loss: 0.3009 - acc: 0.8903 - val_loss: 0.6488 - val_acc: 0.8066 Epoch 47/50 25/25 [=====] - 20s 783ms/step - loss: 0.3078 - acc: 0.8829 - val_loss: 0.6670 - val_acc: 0.7965

Epoch 48/50 25/25 [=====] - 20s 809ms/step - loss: 0.2886 - acc: 0.8906 - val_loss: 0.7584 - val_acc: 0.7828 Epoch 49/50 25/25 [=====] - 20s 797ms/step - loss: 0.3103 - acc: 0.8860 - val_loss: 0.6943 - val_acc: 0.7809 Epoch 50/50 25/25 [=====] - 20s 808ms/step - loss: 0.3007 - acc: 0.8898 - val_loss: 0.7433 - val_acc: 0.7754

Sau khi biên dịch, chúng ta có thể thấy kết quả của mô hình là độ chính xác (accuracy) đạt 88.98% và Độ chính xác xác thực (Val. Acc) là 77.54%, kết quả này có thể được tăng

lên nếu cung cấp thêm dữ liệu để huấn luyện hoặc thực hiện một số thay đổi với mô hình bằng cách thay đổi trình tối ưu hóa (optimizers), hàm kích hoạt hoặc tốc độ học (learning rate).

Bước 8: Xem xét hiệu suất của mô hình

Bây giờ chúng ta sẽ vẽ biểu đồ kết quả mô hình đã huấn luyện của mình với sự trợ giúp của matplotlib.

```
plt.plot (History.history['loss'])  
plt.plot (History.history['val_loss'])  
plt.title('Model Loss')  
plt.ylabel ('Loss')  
plt.xlabel ('Epochs')  
plt.legend (['train', 'test'])  
plt.show()
```

Kết quả: như trong Hình 11.2 (Biểu đồ thể hiện Lỗi (Loss) của Mô hình)

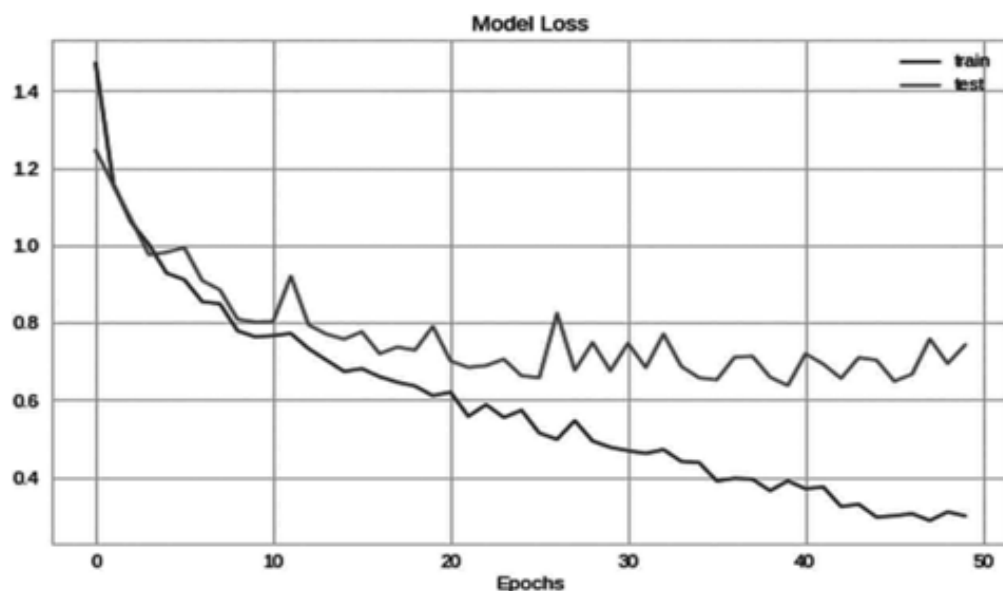


Fig. 11.2 Loss Representation of Model

```
plt.plot (History.history['acc'])  
plt.plot (History.history['val_acc'])  
plt.title('Model Accuracy')  
plt.ylabel ('Accuracy')  
plt.xlabel ('Epochs')  
plt.legend (['train', 'test'])  
plt.show ()
```

Kết quả: như trong Hình 11.3 (Biểu đồ thể hiện Độ chính xác (Accuracy) của Mô hình)

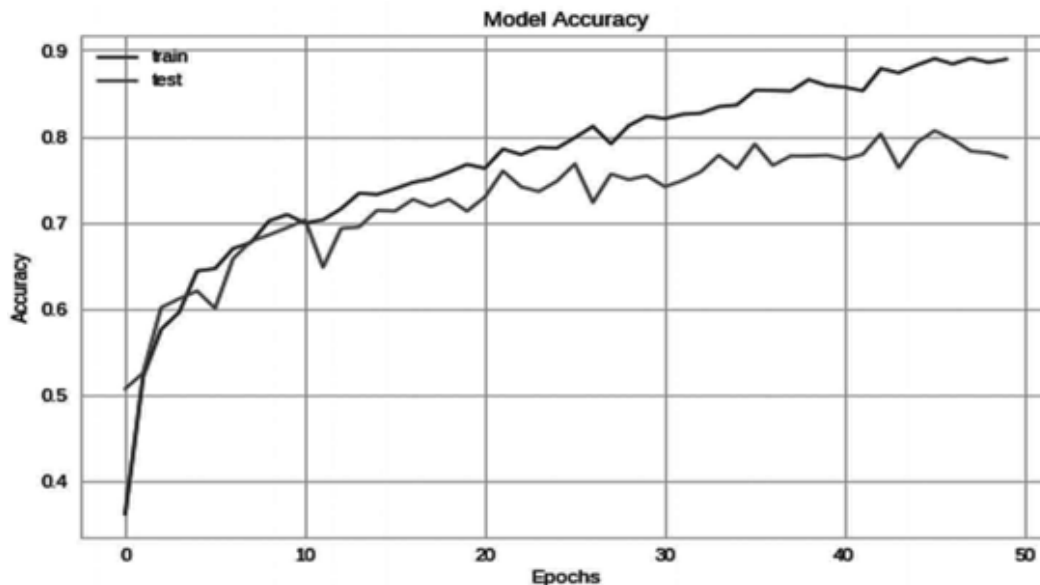


Fig. 11.3 Accuracy Representation of Model

Bước 9: Dự đoán trên dữ liệu xác thực (validation data)

Tại đây, chúng ta sẽ yêu cầu mô hình đã huấn luyện của mình dự đoán các hình ảnh bằng dữ liệu xác thực mà chúng ta đã tách ra trong khi gán nhãn và chia dữ liệu.

```
# lấy các dự đoán trên tập val (xác thực).
```

```
pred = model.predict (x_test)
```

```
pred_digits=np.argmax (pred,axis=1)
```

```
# bây giờ lưu trữ một số chỉ mục (index) được phân loại đúng cũng như phân loại sai.
```

```
i=0
```

```
prop_class=[]
```

```
mis_class=[]
```

```
# Ở đây chúng ta đang nhập các hình ảnh từ dữ liệu xác thực để kiểm tra dự đoán của mô hình.
```

```
for i in range (len (y_test)):
```

```
    if (np.argmax (y_test[i])==pred_digits[i]):
```

```
        prop_class.append (i)
```

```
    if (len (prop_class)==8):
```

```
        break
```

```
i=0
```

```
for i in range (len (y_test)):
```

```
    if (not np.argmax (y_test [i])==pred_digits[i]):
```

```

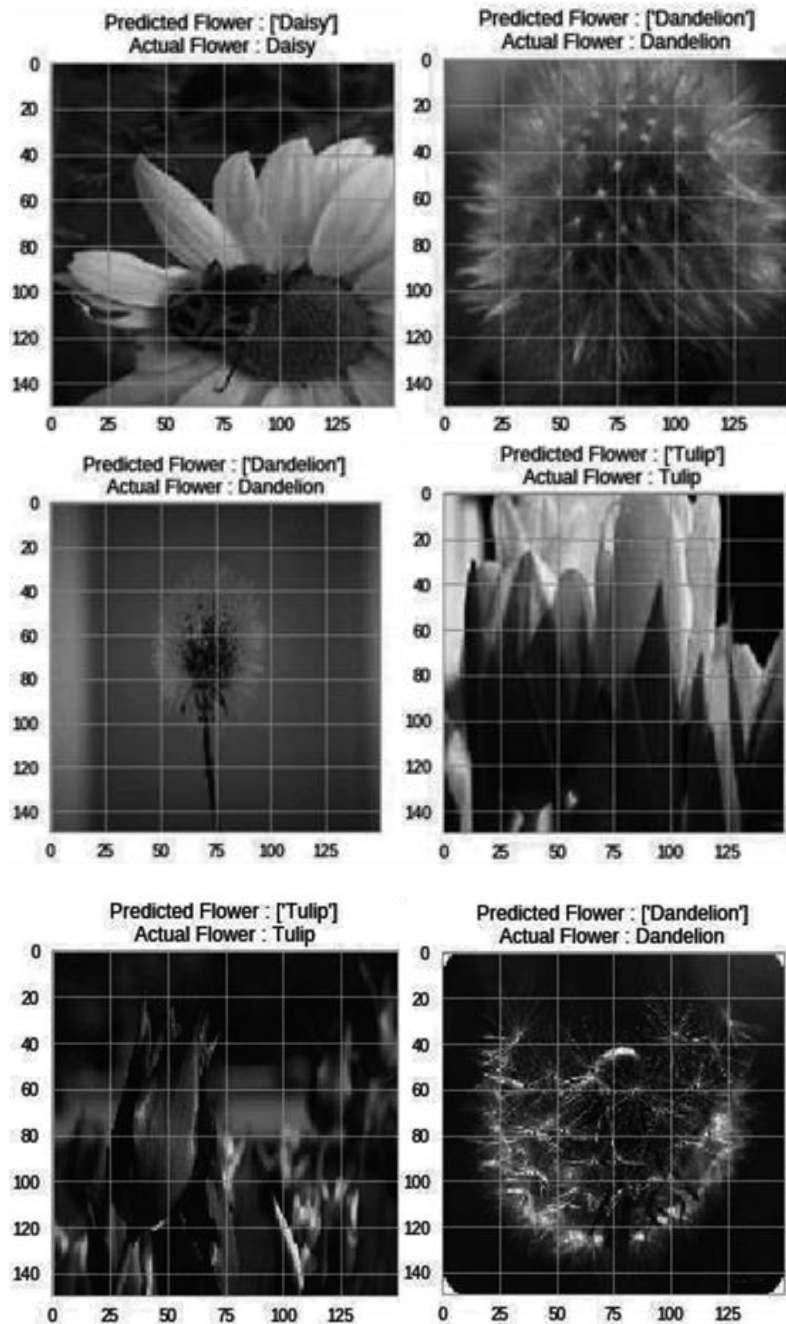
        mis_class.append(i)
    if (len(mis_class)==8):
        break

warnings.filterwarnings('always')
warnings.filterwarnings('ignore')

count = 0
fig,ax=plt.subplots(4,2)
fig.set_size_inches(15,15)
for i in range(3):
    for j in range(2):
        ax[i,j].imshow(x_test[prop_class[count]])
        ax[i,j].set_title("Predicted Flower: "+str(le.inverse_
transform((pred_digits[prop_class[count]])))+ "\n" + "Actual
Flower: "+str(le.inverse_
transform(np.argmax([y_test[prop_class[count]]]))))
    plt.tight_layout()
    count+=1

```

Kết quả: như trong Hình 11.4 (Dự đoán các loài hoa)



Hình 11.4 Dự đoán các loài hoa

Bước 10: Lưu mô hình

Đây là cách để lưu mô hình mà chúng ta đã huấn luyện

```
model_json = model.to_json ()
with open ("model.json", "w") as json_file:
    json_file.write (model_json)
# tuần tự hóa (serialize) trọng số sang HDF5
```



```
model.save_weights("model.h5")  
print ("Model Saved")
```

Sau khi lưu mô hình này, chúng ta có thể dự đoán các giá trị thuộc các danh mục này chỉ bằng cách sử dụng hình ảnh của các loài đó.

Tóm tắt

Nhận dạng loài cũng đóng một vai trò quan trọng trong lĩnh vực nông nghiệp, có thể giúp ích rất nhiều cho mọi người trong việc nhận dạng các loài thực vật bằng kỹ thuật số và nó sẽ tồn tại lâu dài với sự đảm bảo rằng hệ thống của chúng ta là chính xác để nhận dạng một loài thực vật.

Trong chương này, chúng ta đã thực hiện thêm một dự án liên quan đến nhận dạng loài và chúng ta có thể hoàn toàn hiểu cách sử dụng bộ dữ liệu với học máy, học sâu và thị giác máy tính để hiểu rõ hơn và ứng dụng trí tuệ nhân tạo trong thời gian thực.

Trong phần Nhận dạng loài, chúng ta đã sử dụng bộ dữ liệu hoa để huấn luyện mô hình của mình phân loại và nhận dạng các loài hoa trong thời gian thực.

Dự án này có thể được sử dụng trong thời gian thực để nhận dạng các loài hoa bằng kỹ thuật số chỉ từ hình ảnh của hoa.

Tài liệu tham khảo (Bibliography)

Rajesh Singh, Anita Gehlot, Sushabhan Choudhury, Bhupendra Singh, “Embedded System based on ATmega Microcontroller-Simulation, Interfacing and Projects”, Narosa Publishing House, 2017, ISBN: 978-81-8487-5720.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Arduino-Based Embedded Systems: Interfacing, Simulation, and LabVIEW GUI”- CRC Press (Taylor & Francis), 2017, ISBN 9781138060784.

Anita Gehlot, Rajesh Singh, Devender Kumar Saini, Monika Yadav, Bhupendra Singh, “IoT Enabled Smart Microgrid.....Rapid Prototyping”, GBS Publication, 2018, ISBN: 978-93-87374-41-6.

Anita Gehlot, Rajesh Singh, Mamta Mittal, Bhupendra Singh, Ravindra Sharma, Vikas Garg, “Hands on approach on Applications of Internet of Things in various Domain of Engineering”, International Research Publication House, 2018, ISBN: 978-93-87385-25-3.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Internet of Things Enabled Automation in Agriculture”, New India Publishing Agency, 2018, ISBN: 9789387973053.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Bikarama Prasad Yadav, “IoT Enabled Fire Safety and Security Devices for Building”, Pen2Print, EduPedia Publications, 2018, ISBN:9789386647979.

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Piyush Kuchhal, “Wireless Methods and Devices for Home Automation” International Research Publication House, 2018, ISBN: 978-93-87388-27-7.

Rajesh Singh, Anita Gehlot, Raghuveer Chimata, Bhupendra Singh, P.S.Ranjith, “Internet of Things in Automotive Industries and Road Safety” River Publishers, 2018, ISBN:9788770220101&e-ISBN:9788770220095

Rajesh Singh, Anita Gehlot, Bhupendra Singh, Sushabhan Choudhury, “Arduino meets MATLAB. Interfacing, Programs and Simulink”, Bentham Science, 2018, ISBN: 978-1-68108-728-3 & e-ISBN: 978-1-68108-727-6

Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhupendra Singh, and Priyanka Tyagi, “Getting Started for Internet of Things with Launch Pad and ESP8266”, River Publishers, 2019, ISBN: 9788770220682, e-ISBN: 9788770220675

Rajesh Singh, Anita Gehlot, Bhupendra Singh, “Arduino and Scilab based Projects”, Bentham Science, 2019, ISBN: 9789811410918 & e-ISBN: 9789811410925

Anita Gehlot, Rajesh Singh, Lovi Raj Gupta, Bhupendra Singh, “Cook Book for Mobile Robotic Platform Control with Internet of Things and Ti Launch Pad”, BPB Publisher, 2019, ISBN: 978-93-8851-167-4

Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Bhupendra Singh, Mahindra Swain, “Internet of Things with Raspberry Pi and Arduino”, CRC press/Taylor & Francis, 2019, ISBN: 9780367248215

Anita Gehlot, Rajesh Singh, Gaurav Sethi, Bhupendra Singh, “The Robotic Platform Control with Atmega328 and NuttyFi Based on the Internet of Things”, Nova Science Publisher, 2020, ISBN:9781536174724

Rajesh Singh, Anita Gehlot, Lovi Raj Gupta, Navjot Rathour, Mahendra Swain, Bhupendra Singh, “IoT based projects”, BPB publication, 2020, ISBN: 9789389328523